STRIDE Sales in stride.

# IT Deployment Guide

Version 3 · 12 October 2026 · Derby Group · prepared by Fardeen Ismail

## What IT is being asked to set up

STRIDE (the sourcing CRM) is one Node.js service with a single SQLite database file. It needs no database server and no message queue; its one package dependency ships in the archive. IT provides a Linux server inside the bank network, a DNS name with a TLS certificate and the HTTPS reverse proxy in front. Installation takes about an hour. The optional calling bot is a second small service on the same server, described in Step 7; it needs the company's SIP line details and two open-source speech programs.

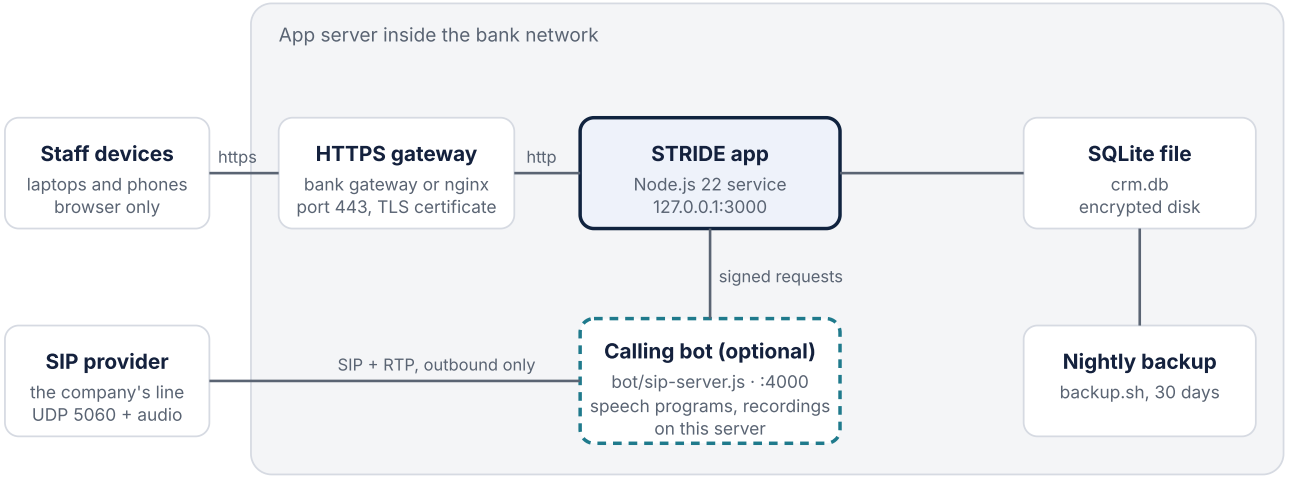
| Item | Requirement |
|---|---|
| Server | Linux VM or container, 2 vCPU, 2 GB RAM, 20 GB disk (the database grows about 1 MB per 1,000 files). With the calling bot: 4 vCPU and 4 GB RAM, as speech recognition runs on the CPU. |
| Runtime | Node.js 22.13 or newer, or Docker Engine 24+ with the compose plugin |
| Network | HTTPS from staff laptops and phones to the server; the server needs no outbound internet. The calling bot needs UDP to the SIP provider only. |
| DNS and TLS | A name such as `crm-test.yourbank.ae` with a certificate the bank's browsers trust |
| Source code | The `fardeen79-ops/crm` repository, branch `claude/sales-sourcing-crm-workflow-rdlw4h`, delivered as an archive, a Git mirror or a Docker image (Step 2) |
| Storage | The database directory on backed-up, encrypted storage; the bot's recordings folder likewise |
| Recording server | Optional: the PBX or call recorder behind the SIP line, reachable from the CRM server either as a read-only mount of its recordings folder or through its HTTP API (Step 7.5). Without it, the call lookup on files shows only the bot's own calls |

HTTPS is mandatory, not optional. The sign-in cookie is marked Secure, and browsers only allow the camera (lead-sheet scanner) and microphone (voice typing) on HTTPS pages.

### Who does what

| Role | Responsibility |
|---|---|
| IT infrastructure | Server, DNS, certificate, reverse proxy, firewall, backups, updates (Steps 1 to 6), the calling bot's line and speech programs (Step 7), the recording server connection (Step 7.5) |
| Business owner (the person requesting this) | Names the first business head and the MIS user, provides the SIP line details, and signs off the acceptance test (Steps 5 and 9) |
| MIS user | Bulk uploads of users, files, targets and card activation inside the app |
| Accounts | Runs the monthly incentive sheets and downloads the incentive payout file; sees nothing else |
| Information security and compliance | Review the security notes before real customer data is entered; approve the caller number and recording notice before bot calls go live |

# Architecture




Deployment architecture: four components, the calling bot and backup optional.

Staff browsers reach the gateway on port 443 only; the gateway forwards to the app on the loopback port, and the app reads and writes one database file. The app makes outbound connections only to alert webhooks, and only when IT configures them. The calling bot talks to the app over signed HTTP on the loopback port and to the SIP provider over UDP; the audio, the speech recognition and the recordings never leave the server.

# Step 1: Prepare the server

One Ubuntu 22.04 or 24.04 LTS server (or RHEL 9) inside the bank network, reachable from staff devices on port 443 only.

1. Create the server and give it the DNS name (for example `crm-test.yourbank.ae`). Confirm the name resolves from a staff laptop.
2. Obtain the TLS certificate and private key for that name from the bank's CA. Place them at `/etc/ssl/certs/<name>.crt` and `/etc/ssl/private/<name>.key`.
3. Firewall: allow inbound 443 (and 80 only for the redirect) from staff networks; allow 22 from the IT admin network. Block everything else inbound. No outbound internet is required. With the calling bot, allow outbound UDP 5060 and UDP 10000–20000 to the SIP provider's addresses only (Step 7).
4. Install git and either Node.js 22 or Docker:

```
# Node.js route (Ubuntu)
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt-get install -y nodejs git sqlite3
node --version        # must print v22.13.0 or newer

# Docker route
sudo apt-get install -y git docker.io docker-compose-v2
sudo systemctl enable --now docker
```

5. Node.js route only: create the service account and folders.

```
sudo useradd -r -s /usr/sbin/nologin crm
sudo mkdir -p /opt/sourcing-crm /var/lib/sourcing-crm /var/backups/sourcing-crm
```

6. Make sure `/var/lib/sourcing-crm` (or the Docker volume) is on encrypted disk and is included in the server backup policy.
7. Set the server clock to sync with NTP. Sales cycles, calling hours and call-back times are computed in UAE time from the server clock.

# Step 2: Getting the code onto the server

The repository is on a personal GitHub account, which the bank server will usually not be allowed to reach. Transfer the code once as a file, then let the server work from a local copy. Pick one of three ways; in Step 3 replace the `git clone` and `git checkout` lines with the matching "on the server" commands below.

### Option 1: Source archive (works for both routes, simplest)

On the personal machine, with Node.js 22 installed:

```
git clone https://github.com/fardeen79-ops/crm.git sourcing-crm
cd sourcing-crm
git checkout claude/sales-sourcing-crm-workflow-rdlw4h
npm ci --omit=dev                      # the one package (needs internet, so do it
here; it goes in the archive)
npm run setup:ocr                      # lead-sheet scanner files (needs internet, so
do it here)
tar czf ../sourcing-crm.tar.gz --exclude=.git --exclude=data --exclude='*.db' --
exclude=.env .
sha256sum ../sourcing-crm.tar.gz       # note the checksum
```

Move `sourcing-crm.tar.gz` (about 25 MB) to the server by the bank's approved method: secure file transfer portal, SFTP from the admin network, or encrypted USB. Then on the server:

```
sha256sum /path/to/sourcing-crm.tar.gz   # must match the checksum noted above
sudo mkdir -p /opt/sourcing-crm
sudo tar xzf /path/to/sourcing-crm.tar.gz -C /opt/sourcing-crm
```

Skip `npm ci` and `npm run setup:ocr` in Step 3; the package and scanner files are already in the archive. Updates arrive as a new archive: stop the service, extract over `/opt/sourcing-crm`, start the service. The database is in `/var/lib/sourcing-crm`, not in the code folder, so it is untouched.

### Option 2: Push to the bank's internal Git (best for ongoing updates)

If IT runs GitLab, Bitbucket or Azure DevOps, create an empty project there and ask for the person's account to have push rights. On the personal machine:

```
git clone --mirror https://github.com/fardeen79-ops/crm.git
cd crm.git
git push --mirror https://git.yourbank.ae/sales/sourcing-crm.git
```

The server then follows Step 3 exactly, with the internal URL in place of the GitHub one. Future updates are a `git push` from the personal machine and a `git pull` on the server. The package and the scanner files still need internet once (`npm ci --omit=dev` and `npm run setup:ocr`), so run those on the personal machine and commit `node_modules` and `public/vendor/tesseract` to the internal repository (both are git-ignored here; use `git add -f`), or transfer those folders with Option 1.

### Option 3: Docker image file (Docker route only)

Build the image on the personal machine and move the finished image, so the server needs neither GitHub nor the Docker Hub base image.

```
# personal machine
git clone https://github.com/fardeen79-ops/crm.git sourcing-crm && cd sourcing-crm
git checkout claude/sales-sourcing-crm-workflow-rdlw4h
docker build -f deploy/Dockerfile -t sourcing-crm:2.0 .
docker save sourcing-crm:2.0 | gzip > sourcing-crm-2.0.tar.gz   # about 80 MB
```

Transfer `sourcing-crm-2.0.tar.gz`, `deploy/docker-compose.yml` and `deploy/.env.example` to the server. On the server:

```
docker load < sourcing-crm-2.0.tar.gz
mkdir -p /opt/sourcing-crm/deploy && cp docker-compose.yml .env.example /opt/sourcing-crm/deploy/
```

Edit `/opt/sourcing-crm/deploy/docker-compose.yml`: delete the `build:` block and its two lines under it, and add `image: sourcing-crm:2.0` in its place. Then run the `docker compose ... up -d` command from Step 3 without `--build`. Each update is a new image file with a new tag.

Whichever option is used, the transferred file contains code only, no customer data. Keep the checksum with the change record so IT can show what was installed.

# Step 3: Install and start the app

Pick one route. Docker is simpler; systemd is for servers where Docker is not approved. Both end with the app listening on `127.0.0.1:3000`, reachable only by the reverse proxy on the same host.

### Route A: Docker

```
git clone https://github.com/fardeen79-ops/crm.git /opt/sourcing-crm
cd /opt/sourcing-crm
git checkout claude/sales-sourcing-crm-workflow-rdlw4h
cp deploy/.env.example deploy/.env
nano deploy/.env                          # fill in: see Step 4
chmod 600 deploy/.env
docker compose -f deploy/docker-compose.yml --env-file deploy/.env up -d --build
docker compose -f deploy/docker-compose.yml logs -f crm   # Ctrl-C when you see "listening"
```

The database lives in the Docker volume `crm-data` (`docker volume inspect deploy_crm-data` shows its path on disk). The image is built from the repository; the only download from a public registry is the `node:22-alpine` base image, which IT can mirror internally if required.

### Route B: systemd (no Docker)

```
sudo git clone https://github.com/fardeen79-ops/crm.git /opt/sourcing-crm
cd /opt/sourcing-crm
sudo git checkout claude/sales-sourcing-crm-workflow-rdlw4h
sudo npm ci --omit=dev                   # the one package (the verification bot's AI
client)
sudo npm run setup:ocr                   # copies the lead-sheet scanner files so they
are served locally
sudo cp deploy/.env.example /etc/sourcing-crm.env
sudo nano /etc/sourcing-crm.env          # fill in: see Step 4
sudo chmod 600 /etc/sourcing-crm.env
sudo chown -R crm:crm /opt/sourcing-crm /var/lib/sourcing-crm
sudo cp deploy/sourcing-crm.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now sourcing-crm
sudo journalctl -u sourcing-crm -f       # Ctrl-C when you see "listening"
```

The unit file runs as the unprivileged `crm` user with `ProtectSystem=strict`, so the process can only write to `/var/lib/sourcing-crm`. Without `npm ci` the CRM still starts; only the verification bot's AI stays off.

**Check it is up** (either route):

```
curl -s http://127.0.0.1:3000/api/me | head -c 200
```

A JSON response means the app is running. The first start creates the database and the first business head account from `ADMIN_EMAIL` and `ADMIN_PASSWORD`.

# Step 4: HTTPS and the reverse proxy

Terminate TLS on the bank's existing web gateway (F5, Citrix ADC, Apache, nginx) and forward to `http://<server>:3000`. If no gateway is available, install nginx on the same server and use the file `deploy/nginx.conf` from the repository.

```
sudo apt-get install -y nginx
sudo cp /opt/sourcing-crm/deploy/nginx.conf /etc/nginx/sites-available/sourcing-crm
sudo nano /etc/nginx/sites-available/sourcing-crm   # set server_name and the two
certificate paths
sudo ln -s /etc/nginx/sites-available/sourcing-crm /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t && sudo systemctl reload nginx
```

Whatever proxy is used, it must:

| Setting | Value | Why |
|---|---|---|

| | | |
|---|---|---|
| Upstream | `http://127.0.0.1:3000` (or the app server's IP on a gateway) | The app itself |
| HTTP to HTTPS | Redirect port 80 to 443 | Sign-in only works on HTTPS |
| TLS versions | TLS 1.2 and 1.3 only | Bank standard |
| Forwarded headers | `Host`, `X-Forwarded-Proto: https`, `X-Forwarded-For` | Correct links and audit log IP addresses |
| Request body limit | 8 MB | Bulk upload spreadsheets and lead-sheet photos |
| Read timeout | 60 seconds | Bulk uploads of a few thousand rows |
| Response headers | `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: no-referrer`, `Strict-Transport-Security: max-age=31536000` | Prevents framing and leaks |

If the app runs on a different host from the gateway, change the Docker port mapping or add a firewall rule so that only the gateway can reach port 3000. Then open `https://crm-test.yourbank.ae` in a browser; the sign-in page should load with a valid padlock and the "Powered by STRIDE" bar at the foot.

# Step 5: Configuration reference

All settings are environment variables in `/etc/sourcing-crm.env` (systemd) or `deploy/.env` (Docker). The file holds a password, so keep it mode 600 and never commit it. Restart the service after any change.

| Variable | Required | Value | Notes |
|---|---|---|---|
| `PORT` | No | `3000` | Port the app listens on behind the proxy |
| `DB_FILE` | Yes | `/var/lib/sourcing-crm/crm.db` | SQLite database file. Docker sets `/data/crm.db` automatically |
| `COOKIE_SECURE` | Yes | `1` | Marks the sign-in cookie Secure. Must be `1` behind HTTPS; sign-in fails otherwise |

| | | | |
|---|---|---|---|
| `ADMIN_NAME`, `ADMIN_EMAIL`, `ADMIN_PASSWORD` | First start | The first business head | Used only when the database has no users. Change the password after first sign-in, then blank it in the file |
| `PUBLIC_URL` | Recommended | `https://crm-test.yourbank.ae` | The address users see, used in alerts and in the links the calling bot answers to |
| `IT_EMAIL`, `IT_EMAIL_WEBHOOK_URL` | Optional | IT mailbox; URL of an email relay that accepts `POST {to, subject, text}` | Approved call-recording requests are emailed to IT. Without it they are only shown in the app |
| `TL_WEBHOOK_URL`, `PROCESSING_WEBHOOK_URL` | Optional | Teams or Slack incoming webhooks | Alerts when a file is marked verification pending, and when a scheduled call-back is due |
| `CALL_BOT_URL`, `CALL_BOT_SECRET` | Optional | `http://127.0.0.1:4000/calls` and a long random secret | Turns on bot calls (Step 7). Blank hides the feature; the bot's teaching page still works |
| `RECORDING_SOURCE` | Optional | `folder` or `http` | Turns on the call lookup on files (Step 7.5). Blank: the lookup lists the bot's calls only |
| `RECORDING_DIR` | With `folder` | `/mnt/recordings` | The PBX's recordings folder, mounted read-only on the CRM server |
| `RECORDING_API_URL`, `RECORDING_API_TOKEN`, `RECORDING_API_MAP` | With `http` | The PBX's list URL with `{number}`, a bearer token, a JSON field map | The CRM calls the API itself and fetches the audio with the token; nothing about the server reaches the browser |
| `RECORDING_LABEL` | Optional | `the 3CX recorder` | How the server is named on screen |
| `ANTHROPIC_API_KEY` | Optional | Leave blank | Lets the verification bot's AI read answers. Compliance must approve first: with it set, customer answers are sent to Anthropic. Blank keeps every call on the server |

With the optional variables blank the app makes no outbound connections at all.

# Step 6: First sign-in and loading users

IT hands over after step 2 below; the business owner and MIS user do the rest inside the app.

1. **IT:** open `https://crm-test.yourbank.ae`, sign in as `ADMIN_EMAIL` with `ADMIN_PASSWORD`. On the Staff page, set a new password for this account.
2. **IT:** on the Staff page, create one user with role **MIS** (name, bank email, HRMS code). Pass the temporary password to that person securely, then blank `ADMIN_PASSWORD` in the environment file.
3. **MIS:** sign in, open **Bulk upload → Users** and upload `docs/go-live/staff-upload.csv` from the archive (433 rows: business heads, sales managers, team leaders, sales staff, processing, governance and staff with no access, with HRMS codes and salaries from the salary sheets). For a fresh list instead, download the template, fill it in and upload. Load sales managers and team leaders first, then sales staff with their core product, so that reporting lines resolve. The upload returns a file of sign-in details to distribute securely.
4. **MIS:** open **Bulk upload → Targets**, download the template, and load targets for the current cycle. Credit card targets are card counts, personal loan and auto loan targets are AED disbursal amounts. Cycles run from the 21st to the 20th and are named after the month they end in.
5. **Business owner:** on the Staff page create the **Accounts** user (role Accounts) and the **CEO** profile if wanted (role CEO). Then have one person from each role (sales, processing, verification team leader, team leader, sales manager, MIS, business head, governance, accounts) sign in and confirm their dashboard loads. Users who forget passwords are reset by MIS or a business head on the Staff page.

**Trying every role before real staff are loaded:** the sample accounts from the demo can be loaded into the live database. On the systemd route run `cd /opt/sourcing-crm && sudo -u crm env $(sudo cat /etc/sourcing-crm.env | grep -v '^#' | xargs) npm run seed:demo`; with Docker run `docker compose exec crm npm run seed:demo`. This adds ten `@demo.local` accounts (team leader, sales manager, two sales staff, processing, verification lead, CEO, accounts, governance, business head), all with password `password123`, and four sample files. Existing users and files are untouched and the command is safe to repeat. Delete the demo accounts on the Staff page before go-live.

Salaries: every profile carries a monthly salary (the upload file has them), counted in the profit and loss. A business head's salary is seen and set only by the CEO profile, the Accounts user or that business head. To load existing files from a spreadsheet, MIS uses **Bulk upload → Cases** with its template. Card activation status is loaded later from **Bulk upload → Cards**.

# Step 7: The calling bot on the company's SIP line (optional)

The verification bot phones customers to confirm the details on a file. It runs as a second service, `bot/sip-server.js`, next to the CRM. Everything about the call stays on this server: it dials over the company's own SIP line, the bot's voice and the recognition of what the customer says are programs installed here, and recordings are files in a folder here. No telephony or speech provider is involved. (A Twilio-based service, `bot/server.js`, exists as an alternative and is not covered here.)

## 7.1 What the business provides

| Item | From |
|---|---|
| SIP server address, username, password | The SIP provider (the company's line). Digest authentication; for a line that authenticates by IP address, the password is blank and registration is switched off |
| Caller number | The number customers see, agreed with compliance |
| Caller name | For example "Verification team" |
| Allowed audio ports | Confirmation from the provider of its SIP and media addresses, for the firewall |

## 7.2 Install the speech programs

Two open-source programs, both CPU-only. Keep them and their models under `/opt/speech`. Neither needs internet after installation.

| Program | Install | Model |
|---|---|---|
| whisper.cpp (recognition) | Build from `github.com/ggml-org/whisper.cpp` (`cmake -B build && cmake --build build --config Release`) on a machine with internet, copy `whisper-cli` to `/usr/local/bin` | `ggml-small.bin` to start (about 470 MB); `ggml-medium.bin` hears better and needs more CPU. Multilingual, so one model covers English and Arabic |
| Piper (voice) | Download the Linux release from `github.com/rhasspy/piper`, copy `piper` and its libraries to `/opt/speech/piper`, link `/usr/local/bin/piper` | A voice per language: `en_GB-alba-medium.onnx` plus its `.onnx.json`; `ar_JO-kareem-medium` for Arabic |

Check each works on its own before touching the CRM:

```
echo "This is a test." | piper --model /opt/speech/en_GB-alba-medium.onnx --
output_file /tmp/t.wav
whisper-cli -m /opt/speech/ggml-small.bin -nt -np -f /tmp/t.wav     # should print:
This is a test.
```

### 7.3 Configure and start the service

```
sudo mkdir -p /var/lib/sourcing-crm/recordings && sudo chown crm:crm
/var/lib/sourcing-crm/recordings
sudo cp /opt/sourcing-crm/deploy/.env.example /etc/sourcing-crm-bot.env
sudo nano /etc/sourcing-crm-bot.env      # the SIP_*, STT_MODEL, TTS_MODEL, BOT_* and
CALL_BOT_SECRET lines
sudo chmod 600 /etc/sourcing-crm-bot.env
sudo cp /opt/sourcing-crm/deploy/sourcing-crm-bot.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now sourcing-crm-bot
sudo journalctl -u sourcing-crm-bot -f   # shows the line registering and the engines
found
```

| Variable | Value |
|---|---|
| `SIP_SERVER`, `SIP_USERNAME`, `SIP_PASSWORD` | From the provider. `SIP_DOMAIN` when the SIP domain differs from the server host; `SIP_REGISTER=0` for IP-authenticated lines |
| `SIP_CALLER_ID`, `SIP_DISPLAY_NAME` | The number and name the customer sees |
| `SIP_LOCAL_IP`, `SIP_MEDIA_IP` | Only behind NAT: the public address the provider must reach, with UDP 5060 and the audio ports forwarded to this server |
| `RTP_PORT_RANGE` | UDP ports for the audio, default `10000-20000`; open them outbound to the provider |
| `STT_MODEL`, `TTS_MODEL`, `TTS_MODEL_AR` | The model paths from 7.2 |
| `BOT_RECORDINGS_DIR`, `BOT_RECORDING_DAYS` | The recordings folder (about 1 MB per minute of call) and how many days to keep them; blank keeps them until deleted |
| `BOT_MAX_CALLS` | Calls at the same time, default 4; each one runs a recognition after every answer, so size it to the CPU |
| `CALL_BOT_SECRET` | The same long random secret as in the CRM's file. It signs requests both ways and is required to serve recordings |
| `ANTHROPIC_API_KEY` | Leave blank to keep the call on the server |

Then set `CALL_BOT_URL=http://127.0.0.1:4000/calls` and the same `CALL_BOT_SECRET` in the CRM's environment file and restart the CRM. If the bot runs on another server, use that host's address and allow port 4000 between the two servers only.

### 7.4 Check before go-live

One command tests the engines, the registration with the provider and, with a number, places a test call that says one line and hangs up. Use the business owner's own phone:

```
cd /opt/sourcing-crm && sudo -u crm env $(cat /etc/sourcing-crm-bot.env | xargs) node bot/sip-check.js --call 0501234567
```

Every line should show a tick. If registration fails, re-check the credentials and the SIP domain; if the call answers but no audio comes back, the audio ports are blocked or `SIP_MEDIA_IP` is wrong. Add `SIP_TRACE=1` to see the exchange with the provider. `curl -s http://127.0.0.1:4000/health` reports the line and the engines at any time.

> **Compliance.** Before the first customer call: the caller number, the recording notice the bot opens with, the calling hours (09:00 to 18:00 in the app) and where recordings are kept. No customer detail leaves the server on this path.

### 7.5 Connecting the recording server (optional)

With this, governance can list every call to or from a customer's number on a file, play it inside the CRM and rate it, once the business head has approved the recording request on that file. The CRM reaches the recording server; browsers never do. Two ways, pick one:

| Route | What IT does |
|---|---|
| Recordings folder | Mount the PBX's recordings share on the CRM server read-only (for example `/mnt/recordings` over SMB or NFS), readable by the `crm` user, and add it to `ReadOnlyPaths` in the service file. Set `RECORDING_SOURCE=folder` and `RECORDING_DIR=/mnt/recordings`. The CRM matches a recording to a number by the digits in the file name, reads the time and direction from the name ( `20261012-093011-971501234567-in.wav` and most other PBX conventions) and the length from the WAV header. WAV, MP3, OGG, M4A, GSM and Opus files are played. |

| | |
|---|---|
| Recording API | Create a read-only API token on the PBX. Set `RECORDING_SOURCE=http`, `RECORDING_API_URL` to the list call with `{number}` where the number goes (digits with country code; `{local}` gives the 0-form, `{e164}` the plus form), `RECORDING_API_TOKEN`, and if the answer's field names differ from the defaults, `RECORDING_API_MAP`, for example `{"list":"data","id":"id","started_at":"start_time","duration":"duration","direction":"direction","extension":"extension","agent":"agent","url":"recording_url"}`. Allow outbound HTTPS from the CRM server to the PBX only. |

Restart the CRM. Test as governance on a completed test file: request the recording, approve it as the business head, press **Look up calls** on the file; the test calls placed to that number appear with a player. Every lookup and listen is in the access log.

# Step 8: Backups, restores, updates and logs

**Backups.** The whole system state is the one database file, plus the bot's recordings folder when bot calls are used. The script `deploy/backup.sh` takes a consistent copy with SQLite's online backup, safe while the app is running, compresses it and deletes copies older than 30 days. Schedule it nightly and ship the copies off the server; add the recordings folder to the server backup policy.

```
# systemd route: run from root's crontab (needs the sqlite3 package)
sudo crontab -e
0 2 * * * DB_FILE=/var/lib/sourcing-crm/crm.db BACKUP_DIR=/var/backups/sourcing-crm
/opt/sourcing-crm/deploy/backup.sh

# Docker route: copy the file out of the volume
docker compose -f /opt/sourcing-crm/deploy/docker-compose.yml exec crm sh -c "cp
/data/crm.db /data/crm-backup.db" && \
  docker cp $(docker compose -f /opt/sourcing-crm/deploy/docker-compose.yml ps -q
crm):/data/crm-backup.db /var/backups/sourcing-crm/crm-$(date +%F).db
```

The backup contains customer names, numbers and salaries. Treat the backup location with the same controls as the live database.

**Restore.** Stop the service, replace the database file with the uncompressed backup, fix ownership, start the service.

```
sudo systemctl stop sourcing-crm
gunzip -c /var/backups/sourcing-crm/crm-20261012-0200.db.gz | sudo tee
/var/lib/sourcing-crm/crm.db > /dev/null
sudo chown crm:crm /var/lib/sourcing-crm/crm.db
sudo systemctl start sourcing-crm
```

**Updates.** Pull the branch (or extract the new archive), reinstall the package, restart. Database changes apply themselves on start; take a backup first.

```
cd /opt/sourcing-crm && sudo git pull && sudo npm ci --omit=dev
sudo systemctl restart sourcing-crm        # and sourcing-crm-bot when it is in use
# Docker route instead:
docker compose -f deploy/docker-compose.yml --env-file deploy/.env up -d --build
```

**Logs and monitoring.**

| Need | Command |
|---|---|
| Live log | `journalctl -u sourcing-crm -f` or `docker compose -f deploy/docker-compose.yml logs -f crm`; the bot: `journalctl -u sourcing-crm-bot -f` |
| Is it up | `curl -s http://127.0.0.1:3000/api/me` returns JSON; `curl -s http://127.0.0.1:4000/health` for the bot. Add both to the monitoring tool, and an HTTPS check on the public URL |
| Disk | Alert when the database directory has less than 2 GB free |
| Service restarts | The units restart the services automatically after a crash; alert on repeated restarts in `journalctl` |

The app logs sign-ins, errors and webhook failures to standard output, which journald or Docker captures. Who viewed or revealed which customer details, and who listened to a bot call, is recorded inside the app under **Access log**, not in the server log.

# Step 9: Acceptance test checklist

Run this on the real server with real devices before the pilot users are invited. Tick each item; the business owner signs off when all are ticked.

### Infrastructure (IT)

- [ ] `https://crm-test.yourbank.ae` loads on a staff laptop and on a staff phone with a valid padlock, with the "Powered by STRIDE" bar at the foot
- [ ] `http://` redirects to `https://`
- [ ] Port 3000 is not reachable from a staff laptop, only through the proxy
- [ ] Service restarts on its own after `sudo kill -9 <pid>` (or `docker kill`)
- [ ] Nightly backup produced a file in the backup folder, and a restore into a scratch copy opens with `sqlite3 ... "select count(*) from users"`
- [ ] The server clock shows UAE time correctly (`timedatectl`)

- [ ] With the calling bot: `sip-check.js --call` rings the business owner's phone, says the test line and hangs up
- [ ] With the recording server: `ls /mnt/recordings` as the `crm` user lists files (folder route), or `curl` with the token returns the call list for a test number (API route)

### Application (business owner and one user per role)

- [ ] First business head signed in and changed the password
- [ ] Users loaded by bulk upload; each role's dashboard opens
- [ ] Targets loaded for the current cycle; sales staff see their own, team leaders see their team's
- [ ] Photograph a lead sheet on My leads from a phone; the lead's details fill in, and Convert to case carries them into the form
- [ ] Voice typing fills a text field, and "Read it back to check" reads it (Chrome, Edge or Safari; not Firefox)
- [ ] One file through the full flow: sales creates, team leader or sales manager approves a flagged file, processing verifies by call, the file completes with a disbursed amount
- [ ] Verification "call back later" set two minutes ahead triggers the call-back alert and, if configured, the Teams or Slack message
- [ ] Changing the product, loan amount, interest rate or FPD after verification sends the file back for re-verification
- [ ] Phone and salary show masked; reveal works, re-hides after 3 minutes, and the reveal appears in the Access log for governance
- [ ] A screenshot of any page shows the user's name and time watermark
- [ ] Credit card completed files appear as inactive with ageing; bulk card activation upload marks them active with a date
- [ ] A personal loan completed with a rate shows on the sales person's, team leader's and business head's yield card against the 8.49% minimum
- [ ] Chat: a case discussion and a direct message deliver, and governance can read them
- [ ] Calls to this customer: locked on a file until governance requests the recording and the business head approves; then the calls list, play inside the CRM, and governance's rating shows on the file's history
- [ ] Accounts user sees only Reports: the incentive sheets run for the cycle, grouped by region and hierarchy, and the Incentive payout file downloads with HRMS codes
- [ ] A downloaded case register has no phone column; a business head's salary reads Hidden to MIS on the Staff page
- [ ] Bulk upload of files from the template with the bank's own test spreadsheet
- [ ] With the calling bot: the verification team leader asks for a bot call on a test file, governance and the business head approve, the call reaches the test phone, and the result, transcript and recording appear on the file

# Security notes and production go-live

The test environment is suitable for a pilot with a small group and test or consented customer data. The items below are required before the system becomes the system of record, and information security should review them now.

## Already built in

- Passwords hashed with scrypt; sessions in HttpOnly, Secure, SameSite cookies; all writes require JSON bodies, which blocks cross-site form posts
- Role-based access: each role sees only its own files, team or scope; a business-defined roles register for custom roles
- No Emirates ID or passport numbers are stored: the form has no such fields and earlier values were wiped on upgrade
- Phone numbers and salaries masked on screen; every reveal, file view, report run and bot-call listen logged with user, time and IP in the Access log, visible to governance and business heads
- Watermark of the viewer's name, code and time on every page
- The calling bot's requests and results are signed with a shared secret; recordings are served only to the CRM with a signed, time-limited request
- Processes run as an unprivileged user with a read-only filesystem apart from the data directory; one package dependency (Anthropic's SDK) to keep patched

## Required before production go-live

| Item | Why | Effort |
|---|---|---|
| Single sign-on with the bank directory (SAML or OpenID Connect) | Joiners, leavers and password policy follow the bank's | Medium: an identity provider endpoint from IT, then an app change |
| Login lockout and password rules | Brute-force protection | Small app change |
| Session timeout of 15 to 30 minutes idle | Shared devices | Small app change |
| Penetration test | Bank policy for any system holding customer data | Scheduled with the security team |
| Log forwarding to the bank SIEM | Central audit | IT: forward journald and the Access log export |
| Data retention policy | Files, chat messages, access logs and call recordings grow indefinitely | Decision from compliance, then `BOT_RECORDING_DAYS` and a scheduled purge |

| | | |
|---|---|---|
| Compliance sign-off on bot calls | Automated calls to customers, the caller number, the recording notice | Before the first live call; the AI stays off unless separately approved |

### Keep in mind during the pilot

- Screenshots and phone photos of the screen cannot be blocked by a web application. The watermark and access log are deterrents and evidence, not prevention.
- Voice typing uses the browser's own speech engine. On Chrome and Edge that audio goes to Google or Microsoft; nothing is sent to the CRM server. If that is not acceptable, disable the microphone in the browser policy and voice typing will be hidden. The calling bot does not use the browser's engine: its recognition runs on the server.

# Troubleshooting

| Symptom | Cause | Fix |
|---|---|---|
| Service exits at once; log says `node:sqlite` or `DatabaseSync` is not available | Node.js older than 22.13 | Install Node.js 22.13 or newer and restart |
| Sign-in page loads but signing in returns to the same page | `COOKIE_SECURE=1` while the site is reached over plain HTTP, or the proxy does not pass `X-Forwarded-Proto` | Use HTTPS end to end; check the proxy headers in Step 4 |
| `SQLITE_READONLY` or `EACCES` in the log | Data directory not writable by the `crm` user, or not listed in `ReadWritePaths` | `chown -R crm:crm /var/lib/sourcing-crm`; keep the unit file's `ReadWritePaths` |
| `EADDRINUSE: 3000` | Another process on port 3000 | Stop it, or set `PORT` and the proxy upstream to another port |
| Bulk upload fails with `413` | Proxy body limit too small | Set the proxy limit to 8 MB |
| Lead-sheet scanner says it cannot load | Scanner files not installed (systemd route) | Run `npm run setup:ocr` in `/opt/sourcing-crm` and restart |
| Camera or microphone buttons do nothing | Page not on HTTPS, or the browser blocked the device | Check the padlock; allow camera and microphone for the site in the browser |
| Teams or Slack alert not received | Webhook URL wrong, or outbound HTTPS blocked from the server | Log shows `webhook failed`; test with `curl -X POST <url> -d '{"text":"test"}'` from the server |

| | | |
|---|---|---|
| "Call with bot" missing on files | `CALL_BOT_URL` blank in the CRM's file | Set it and restart the CRM |
| Bot call fails at once: "The SIP line is not registered" | Wrong credentials or SIP domain, or the provider unreachable on UDP 5060 | Run `sip-check.js` with `SIP_TRACE=1`; check the firewall towards the provider |
| Bot call connects but the customer hears nothing, or the bot hears nothing | Audio ports blocked, or `SIP_MEDIA_IP` wrong behind NAT | Open UDP 10000–20000 to the provider; set the public address in `SIP_LOCAL_IP` and `SIP_MEDIA_IP` and forward the ports |
| Bot result says "The bot's voice could not be produced" or every answer is not understood | Piper or whisper-cli not on the PATH, or a model path wrong | Run the 7.2 commands as the `crm` user; check `STT_MODEL` and `TTS_MODEL` |
| Bot calls slow to answer a question | CPU too small for the recognition model, or too many calls at once | Use `ggml-small.bin`, lower `BOT_MAX_CALLS`, or add CPU |
| "Look up calls" says the server could not be reached, or lists nothing | Share not mounted or not readable by `crm`; API URL, token or field map wrong; outbound HTTPS to the PBX blocked | Folder: `sudo -u crm ls $RECORDING_DIR`. API: `curl -H "Authorization: Bearer $TOKEN" "$URL"` with a number in place of `{number}`; the answer must carry the list named in the map |
| Calls card says the approvals are not in place | By design | Governance requests the recording on the file (verification must be complete), the business head approves; the card then unlocks |
| Call-back alert late | Server clock wrong | Enable NTP; alerts are checked every 30 seconds in UAE time |
| Admin password forgotten and no other business head | Only the first start creates the admin | Stop the service, move the database file aside, start once with a new `ADMIN_PASSWORD`, then restore the database and have the original admin reset others, or ask the developer for a reset script |
| Database file growing fast | Chat messages and access log | Expected; about 1 MB per 1,000 files plus logs. Plan retention per the security notes |

For anything else, send the last 100 lines of `journalctl -u sourcing-crm` (or the Docker log) with the time of the error.